RTL Design and Verilog Coding Guidelines

Professional Guidelines for RTL Design Engineers

October 12, 2026

Contents

1 Introduction **3**

2 Use Meaningful Names and Modular Design **3**

3 Combinational Logic Coding **4**
3.1 Correct Example 4
3.2 Incorrect Example 4

4 Sequential Logic Coding **4**

5 Blocking vs. Nonblocking Assignments **5**
5.1 Example: Register Pipeline 5

6 Reset Design Guidelines **5**
6.1 Synchronous Reset 5
6.2 Asynchronous Reset 6

7 Finite State Machine (FSM) Design **6**

8 Case Statements and Conditional Logic **7**

9 Bit Width and Signedness **8**
9.1 Example 8

10 Parameterized and Reusable RTL **8**

11 Generate Constructs **9**

12 Synthesizability Guidelines **9**

13 Clocking and Timing Guidelines **10**

14 Clock-Domain Crossing (CDC) **10**
14.1 Two-Flip-Flop Synchronizer 10

15 Avoid Multiple Drivers **11**

16 Lint, Simulation, and Synthesis Checks **11**

17 Common RTL Coding Mistakes **12**

18 Final Summary **12**

1 Introduction

RTL (Register Transfer Level) design describes how data moves between registers and how combinational logic transforms that data.

An RTL design engineer must write code that is:

- Functionally correct and synthesizable.
- Free from unintended latches and multiple drivers.
- Predictable during simulation.
- Suitable for timing closure and physical implementation.
- Easy to verify, maintain, and reuse.

Good RTL coding practices improve design quality and reduce simulation, synthesis, and hardware debugging effort.

2 Use Meaningful Names and Modular Design

Use descriptive names for modules, ports, registers, parameters, and internal signals.

```
module data_register #(
    parameter DATA_WIDTH = 8
)(
    input  wire                  clk,
    input  wire                  rst,
    input  wire [DATA_WIDTH-1:0] data_in,
    output reg  [DATA_WIDTH-1:0] data_out
);

    always @(posedge clk) begin
        if (rst)
            data_out <= {DATA_WIDTH{1'b0}};
        else
            data_out <= data_in;
    end

endmodule
```

Guidelines:

- Use descriptive names such as `data_out` instead of unclear names such as `x1`.
- Divide complex designs into smaller functional modules.
- Use parameters for configurable widths and depths.
- Define clear module interfaces and responsibilities.

3 Combinational Logic Coding

Combinational logic produces outputs based only on current inputs.

In Verilog, use `always @(*)` for combinational procedural logic. In SystemVerilog, prefer `always_comb`.

3.1 Correct Example

```
always @(*) begin
    y = 1'b0;

    if (enable)
        y = a & b;
end
```

The output `y` is assigned on every execution path, so this code does not infer a latch.

3.2 Incorrect Example

```
always @(*) begin
    if (enable)
        y = a & b;
end
```

When `enable` is zero, `y` retains its previous value. This behavior may infer a latch.

Guidelines:

- Assign every output on every possible path.
- Provide default assignments at the beginning of combinational blocks.
- Avoid unintended storage elements.
- Avoid combinational feedback unless explicitly required and carefully designed.

4 Sequential Logic Coding

Sequential logic stores data and changes state at specified clock edges.

Use nonblocking assignments (`<=`) in clocked procedural blocks.

```
always @(posedge clk) begin
    if (rst)
        q <= 1'b0;
    else
        q <= d;
end
```

Here, `q` captures `d` at the rising clock edge unless reset is asserted.

Guidelines:

- Use nonblocking assignments for registers.
- Keep clocked processes easy to understand.
- Avoid multiple procedural blocks driving the same register.
- Do not use combinational assignments to update a register intended to be clocked.

5 Blocking vs. Nonblocking Assignments

Feature	Blocking (=)	Nonblocking (<=)
Update behavior	Immediate procedural update	Schedules update for the NBA region
Typical usage	Combinational logic	Clocked sequential logic
Main concern	Ordering can cause races	Models simultaneous register updates

5.1 Example: Register Pipeline

```
always @(posedge clk) begin
    stage1 <= data_in;
    stage2 <= stage1;
end
```

Both right-hand sides are evaluated using the values available when the statements execute. The scheduled updates occur in the NBA region.

Therefore, `stage2` receives the previous value of `stage1`, correctly modeling a two-stage pipeline.

6 Reset Design Guidelines

Reset initializes registers to known values. The reset strategy must match the design specification.

6.1 Synchronous Reset

A synchronous reset is checked only at the active clock edge.

```
always @(posedge clk) begin
    if (rst)
        q <= 1'b0;
    else
        q <= d;
end
```

6.2 Asynchronous Reset

An asynchronous reset can affect the register without waiting for a clock edge.

```
always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        q <= 1'b0;
    else
        q <= d;
end
```

Here, `rst_n` is an active-low reset.

Guidelines:

- Follow the specified reset polarity.
- Use a consistent reset strategy within each related group of registers.
- Ensure all required registers reach valid states after reset.
- For asynchronous resets, consider safe deassertion and recovery/removal timing.

7 Finite State Machine (FSM) Design

An FSM consists of states, transition logic, and outputs. It is commonly used in controllers and protocol logic.

```
localparam IDLE  = 2'b00;
localparam WAIT  = 2'b01;
localparam DONE  = 2'b10;

reg [1:0] state, next_state;

always @(posedge clk) begin
    if (rst)
        state <= IDLE;
    else
        state <= next_state;
end

always @(*) begin
    next_state = state;

    case (state)
        IDLE: begin
            if (start)
                next_state = WAIT;
        end

        WAIT: begin
            if (ready)
                next_state = DONE;
```

```
        end

        DONE: begin
            next_state = IDLE;
        end

        default: next_state = IDLE;
    endcase
end
```

Guidelines:

- Separate state registers from transition logic.
- Use named state constants instead of unexplained numeric values.
- Provide default assignments in combinational logic.
- Include a suitable default state transition.
- Choose state encoding according to design and synthesis requirements.

A default branch alone does not prevent latches if other combinational outputs are not assigned on every path.

8 Case Statements and Conditional Logic

Use complete and clearly defined decision logic.

```
always @(*) begin
    case (sel)
        2'b00: y = a;
        2'b01: y = b;
        2'b10: y = c;
        2'b11: y = d;
        default: y = 1'b0;
    endcase
end
```

Guidelines:

- Cover all required input combinations.
- Include a default branch where appropriate.
- Avoid `casex` in synthesizable RTL because it treats unknown and high-impedance bits as wildcards.
- Use `casez` only when wildcard behavior is intentional and safe.
- Check that assignments are complete to avoid unintended latches.

9 Bit Width and Signedness

Width mismatches and signedness errors can cause truncation, incorrect arithmetic, and unexpected comparisons.

9.1 Example

```
reg [7:0] a, b;
reg [7:0] sum;

always @(*) begin
    sum = a + b;
end
```

The result is stored in an 8-bit signal. If the addition exceeds 255, the carry beyond bit 7 is lost.

To preserve the carry, use a 9-bit result.

```
reg [7:0] a, b;
reg [8:0] sum;

always @(*) begin
    sum = {1'b0, a} + {1'b0, b};
end
```

Guidelines:

- Define appropriate widths for arithmetic results.
- Understand signed and unsigned expressions.
- Use explicit sizing for important constants.
- Check shifts, comparisons, extensions, and truncation.

10 Parameterized and Reusable RTL

Parameters allow a module to support multiple hardware configurations.

```
module adder #(
    parameter WIDTH = 8
)(
    input  wire [WIDTH-1:0] a,
    input  wire [WIDTH-1:0] b,
    output wire [WIDTH-1:0] sum
);

    assign sum = a + b;

endmodule
```

Guidelines:

- Use parameters for configurable widths and depths.
- Use `localparam` for internal derived constants.
- Check parameter-dependent vector widths.
- Use `$clog2` carefully, particularly when calculating address widths for a depth of one.
- Test multiple legal parameter configurations.

11 Generate Constructs

Generate constructs create repetitive hardware structures during elaboration.

```
module and_array #(
    parameter WIDTH = 8
)(
    input  wire [WIDTH-1:0] a,
    input  wire [WIDTH-1:0] b,
    output wire [WIDTH-1:0] y
);

    genvar i;

    generate
        for (i = 0; i < WIDTH; i = i + 1) begin
            assign y[i] = a[i] & b[i];
        end
    endgenerate

endmodule
```

The loop creates one AND gate per bit.

Guidelines:

- Use generate loops for repeated hardware structures.
- Ensure loop bounds and conditions are elaboration-time constants where required.
- Avoid unnecessary duplication of RTL.

12 Synthesizability Guidelines

Not every Verilog construct represents synthesizable hardware.

Common simulation-only constructs include:

- Delay controls such as `#10` in RTL logic.
- Testbench tasks and stimulus generation.
- System tasks such as `$display`.

- File operations used only for simulation.

For example:

```
// Testbench-style delay; not ordinary synthesizable RTL
initial begin
    #10;
    signal = 1'b1;
end
```

Use clocked logic and appropriate counters or timers to represent hardware delays.

Synthesis support depends on the construct, coding context, and tool. Check the synthesis tool's supported RTL subset.

13 Clocking and Timing Guidelines

Clocking structure directly affects timing closure.

- Use the intended clock for each sequential block.
- Avoid generating clocks using ordinary combinational logic or arbitrary RTL expressions.
- Use proper clock enables when appropriate.
- Keep combinational paths short enough to meet timing constraints.
- Avoid unnecessary logic depth between registers.
- Review setup and hold timing reports after synthesis and implementation.

A functionally correct simulation does not guarantee that the circuit will meet its clock frequency.

14 Clock-Domain Crossing (CDC)

A clock-domain crossing occurs when a signal passes between circuits driven by unrelated or asynchronous clocks.

Improper CDC logic can cause metastability or data corruption.

14.1 Two-Flip-Flop Synchronizer

For a single-bit control signal, a common approach is a two-stage synchronizer.

```
(* ASYNC_REG = "TRUE" *) reg sync1, sync2;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        sync1 <= 1'b0;
        sync2 <= 1'b0;
    end
    else begin
```

```
        sync1 <= async_in;
        sync2 <= sync1;
    end
end
```

The destination logic uses `sync2`, not the raw asynchronous input.

Important: A two-flip-flop synchronizer is appropriate for many single-bit level signals, but it is not a universal solution for multi-bit buses, pulses, or data transfers. Use suitable CDC protocols, such as handshakes or asynchronous FIFOs, when needed.

15 Avoid Multiple Drivers

Each register or variable should normally have one clearly defined procedural owner.

Incorrect design:

```
always @(posedge clk)
    q <= a;

always @(posedge clk)
    q <= b;
```

Both processes attempt to update `q`, making the intended behavior unclear and potentially causing simulation or synthesis problems.

Combine the logic into one process.

```
always @(posedge clk) begin
    if (select_a)
        q <= a;
    else
        q <= b;
end
```

16 Lint, Simulation, and Synthesis Checks

Professional RTL development requires more than writing code.

1. **Lint:** Detect coding violations, width mismatches, unused signals, and suspicious logic.
2. **Simulation:** Verify functional behavior using directed and constrained-random tests as appropriate.
3. **Synthesis:** Check inferred registers, latches, memories, and logic structures.
4. **Static timing analysis:** Verify setup and hold requirements against design constraints.
5. **CDC analysis:** Identify unsafe crossings between clock domains.

6. **Code review:** Check readability, maintainability, and consistency with project standards.

Resolve important warnings rather than ignoring them without justification.

17 Common RTL Coding Mistakes

Mistake	Recommended Practice
Blocking assignments in clocked logic	Prefer nonblocking assignments for registers.
Incomplete combinational assignments	Assign outputs on every possible path.
Multiple procedural writers	Use one clear procedural owner per register.
Unintended width truncation	Size arithmetic results correctly.
Unsynchronized asynchronous inputs	Use appropriate synchronizers or CDC protocols.
Simulation delays in synthesizable logic	Use clocked hardware structures.
Unverified parameter combinations	Test all supported configurations.
Ignoring timing warnings	Review timing constraints and reports.

18 Final Summary

A professional RTL design engineer should follow these principles:

- Write clear, modular, and synthesizable RTL.
- Use blocking assignments for combinational procedural logic and nonblocking assignments for clocked sequential logic.
- Avoid unintended latches, multiple drivers, and simulation races.
- Follow consistent reset and FSM design practices.
- Check widths, signedness, and parameter values.
- Apply proper clocking and CDC techniques.
- Verify functionality through simulation and lint.
- Check synthesis results and timing reports.

Key takeaway: Good RTL is not merely code that simulates correctly. It must also synthesize into the intended hardware, meet timing requirements, and remain readable and maintainable throughout the design lifecycle.